

14.2

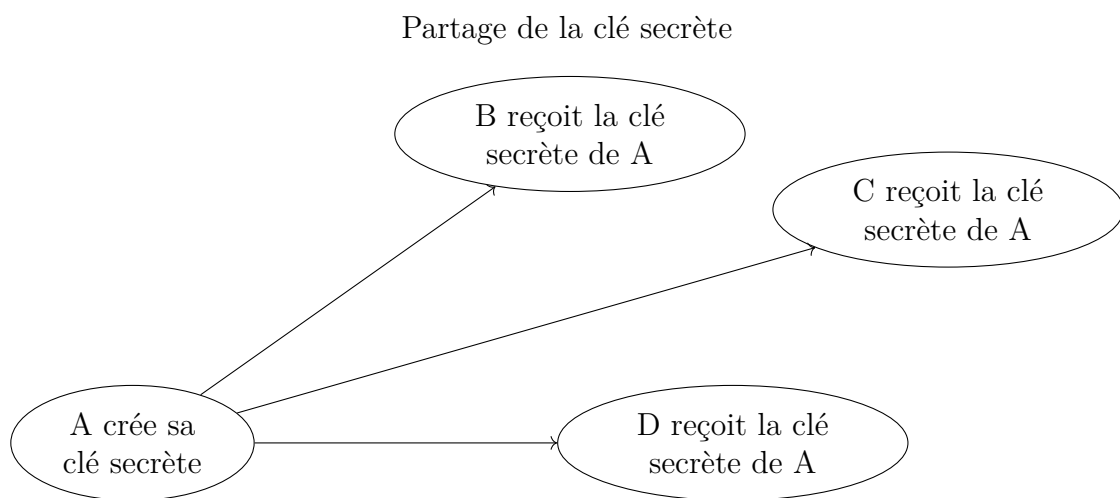
Le chiffrement symétrique ou chiffrement à clef partagée

NSI TERMINALE - JB DUTHOIT

14.2.1 Principe du chiffrement symétrique

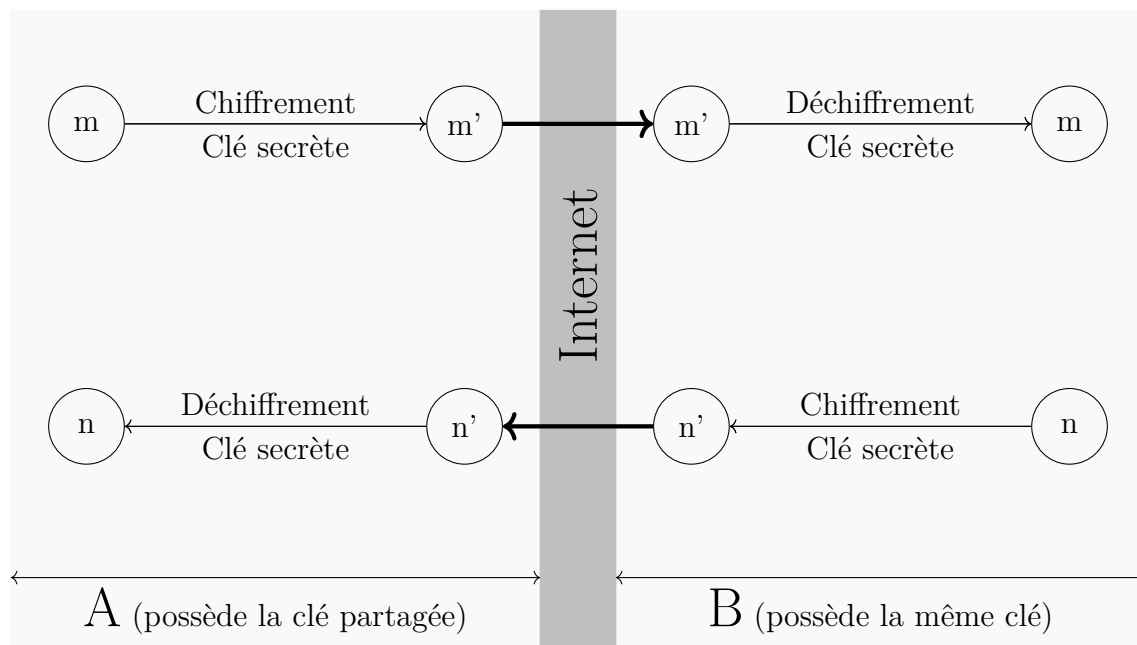
Supposons que (A) ait l'envie d'envoyer un message à (B), (C) et (D) de façon sécurisée.

(A) doit créer une clé de chiffrement (clé secrète) et la transmettre à (B), (C) et (D) :



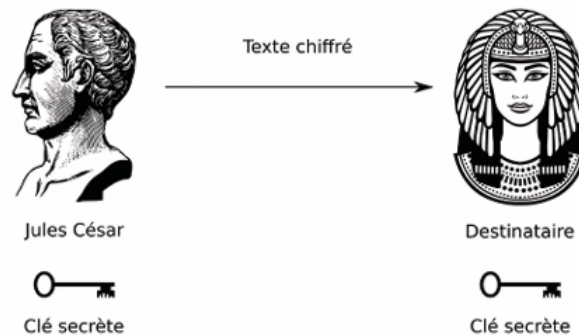
La clé va servir à chiffrer lors de l'envoi et à déchiffrer lors de la réception :

Envoi et réception d'un message



14.2.2 Le chiffrement de César

L'une des méthodes les plus anciennes pour chiffrer un texte est attribuée à Jules César. Elle consiste à remplacer chaque lettre du message par la lettre située trois places plus loin dans l'alphabet. Par extension, on continue à appeler cette technique un chiffrement de César même si le décalage vaut autre chose que 3. On appelle le premier message le clair et celui obtenu après décalage le chiffré.



Pour simplifier, on considère dans toute cette activité qu'on ne chiffre que les lettres majuscules non accentuées, les autres caractères (espaces inclus) restant inchangés.

Exercice 14.1

Les lettres sont des nombres cachés ! Pour que l'ordinateur puisse décaler des lettres (comme dans le code de César), il faut d'abord qu'il transforme ces lettres en nombres. En Python, chaque caractère possède un code numérique.

1. Découverte de `ord()` et `chr()`

Ouvre la console Python (le shell) et teste les commandes suivantes :

- Tape `ord("A")`, puis `ord("B")`, puis `ord("C")`. Que remarques-tu ? Quel est le code de la lettre "A" ?
- La fonction inverse est `chr()`. Tape `chr(65)`, puis `chr(66)`. Comment faire pour afficher la lettre "Z" avec cette fonction ?

2. Convertir des lettres en nombres et inversement

Complète la fonction suivante pour qu'elle renvoie la position de la lettre dans l'alphabet (de 0 à 25) :

```
def lettre_vers_nombre(lettre):
    code = ord(lettre)
    position = ...
    return position
```

Tests à valider :

```
print(lettre_vers_nombre("A")) # Doit afficher 0
print(lettre_vers_nombre("C")) # Doit afficher 2
```

Complète la fonction suivante qui renvoie la lettre associée au nombre entré en paramètre :

```
def nombre_vers_lettre(position):
    code = ...
    lettre = chr(code)
    return lettre
```

Tests à valider :

```
print(nombre_vers_lettre(0)) # Doit afficher A
print(nombre_vers_lettre(25)) # Doit afficher Z
```

Exercice 14.2

On s'intéresse ici au chiffrement de César, avec uniquement des lettres majuscules.

1. En considérant un décalage de 3, chiffrer le mot **TERMINALE**.
2. Quelle opération faut-il effectuer pour déchiffrer un message ? Déchiffrer alors le mot **LQIRUPDWLTXH** (toujours avec $k = 3$).
3. Créer une fonction python `chiffrement_cesar(k,mot)` qui effectue un chiffrement de César avec un décalage de k lettres sur `mot`. La fonction renvoie le mot chiffré.
4. Créer une fonction python `dechiffrement_cesar(k,mot_chiffre)` qui effectue un déchiffrement de `mot_chiffre`, sachant que `mot_chiffre` a été chiffré avec un chiffrement de César avec un décalage de k lettres. La fonction renvoie le mot déchiffré.

Exercice 14.3

Utiliser la force brute pour déchiffrer 'ZCWRLKKIRMRZCCVIGCLJ' message chiffré avec un chiffrement de César.

14.2.3 Le chiffrement de Vignère

Ici, la clef n'est pas un entier, mais un mot :

Choisissons donc maintenant un "mot" qui nous servira de clé de chiffrement. Je choisis par exemple **JB** :

On va ensuite "recopier" autant de fois que nécessaire la clé pour avoir exactement autant de lettres que le mot à coder :

mot à coder	B	O	N	J	O	U	R
clé	J	B	J	B	J	B	J

Ensuite, nous allons coder la lettre **B** avec un décalage de 9 (pour la lettre **J**), la lettre **O** avec un décalage de 1 (pour la lettre **B**), la lettre **N** avec un décalage de 9 ...etc...

mot à coder	BONJOUR
clé	JB JB JB
mot chiffré	

Exercice 14.4

Pour chiffrer un texte très long, la méthode consistant à recopier la clé "à la main" autant de fois que nécessaire devient vite fastidieuse, et occupe inutilement de la mémoire en informatique. L'objectif de cet exercice est de découvrir une méthode mathématique plus directe, très utile pour programmer. On souhaite chiffrer un message avec la clé **OUI** (de longueur 3). En informatique, on numérote souvent la position des lettres d'un mot en commençant par l'indice 0. Pour notre clé "OUI", les indices sont donc : 0 pour 'O', 1 pour 'U', et 2 pour 'I'.

1. Afin de bien comprendre comment se répète la clé, compléter le tableau suivant pour les six premières lettres du texte à chiffrer :

Indice de la lettre du texte	0	1	2	3	4	5
Lettre de la clé correspondante	O					
Indice de cette lettre dans la clé	0					

2. Sans écrire toute la suite, quelle lettre de la clé sera utilisée pour chiffrer la lettre d'indice 14 du texte à coder ? Expliquer le calcul.
3. Quelle lettre de la clé qui servira à chiffrer la lettre située à l'indice 1 000 du message ?
4. De manière générale, si une lettre du texte se trouve à l'indice i , quelle opération mathématique permet de déterminer directement l'indice de la lettre de la clé à utiliser ?

Exercice 14.5

Créer une fonction `chiffrement_vignere(mot,cle)` qui prend en argument une chaîne de caractères `mot` et une chaîne de caractère `cle`, et qui renvoie le mot `mot` codé avec le chiffrement de Vignère avec la clé `cle`.

Exercice 14.6

Créer une fonction `dechiffrement_vignere(mot,cle)` qui déchiffre un mot qui a été chiffrer avec le chiffrement de Vignère avec la clé `cle`

Exercice 14.7

On utilise la méthode de chiffrement de Vignère aux 95 caractères ASCII utilisables.

1. Vérifier qu'il existe environ cent million de clefs différentes comportant 4 caractères
2. Si l'on peut tester 1000 clefs par seconde, combien de temps est nécessaire pour déchiffrer un message par recherche exhaustive ?

14.2.4 Le chiffrement de Vernam

Comme le chiffrement de César, le chiffrement de Vignère a une sécurité très faible : à cause de la répétition de la clé, on peut utiliser l'analyse de fréquences (test de Kasiski) pour trouver la longueur de la clé et casser le code.

Inventé par Gilbert Vernam (et perfectionné par Claude Shannon), c'est le seul système de chiffrement prouvé mathématiquement comme étant **incassable**.

Fonctionnement : C'est un Vigenère, mais avec des conditions ultra-strictes sur **La Clé qui doit être :**

1. Aussi **longue** que le message lui-même.
2. Totalelement **aléatoire**.
3. Utilisée **une seule fois** (jetable).

Sécurité : Absolue. Si vous interceptez le message, chaque lettre a une probabilité égale d'être n'importe quelle autre lettre. Sans la clé, le texte chiffré ne contient aucune information.

14.2.5 Le chiffrement XOR

Jusqu'ici, nous avons manipulé des lettres ($A, B, C \dots$) et des décalages alphabétiques. Mais comment un ordinateur, qui ne comprend que des 0 et des 1, chiffre-t-il une information ? Il utilise une opération logique universelle : le XOR (OU exclusif). Le XOR est au monde numérique ce que le chiffre de César était à l'Antiquité, avec une différence magique : alors que César nécessite de "faire plus" pour chiffrer et "faire moins" pour déchiffrer, le XOR est auto-inverse.

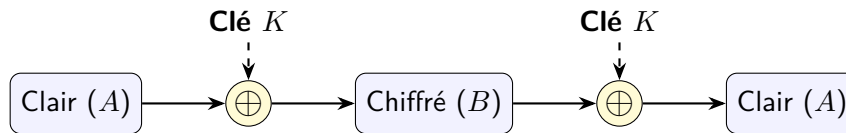


Schéma de la réversibilité du chiffrement XOR

Analysez et testez ce programme :

```
def chiffrage(message, cle):
    mess_code = ""
    position = 0
    for lettre in message:
        mess_code = mess_code + chr(ord(lettre) ^ cle[position])
        position = (position + 1) % len(cle)
    return mess_code
```

Remarque

- Ici, la clé est exprimée sous la forme d'un tableau d'entiers et l'opérateur "^" permet de faire un ou exclusif bit à bit. Par exemple :
66 ^ 68 donne 6 car 66 s'écrit 1000010 et 68 1000100 donc l'"opération" ou exclusif bit à bit donne 0000110, soit le nombre 6...
- Les caractéristiques de l'opérateur XOR, et le fait qu'il puisse être implémenté directement dans le processeur, font qu'il est souvent utilisé dans les algorithmes de chiffrement modernes, comme AES ou ChaCha20

Exercice 14.8

Utilisez les fonctions précédentes pour chiffrer la phrase `Veuillez transférer 1000 euros sur mon compte off-shore` en utilisant la clef `ũšĜtŪkĜuŽũkŴŪũũŵt.`. Cette clef s'obtient en considérant les caractères dont l'unicode est [365, 353, 288, 355, 364, 489, 288, 371, 377, 365, 489, 372, 370, 361, 369, 373, 357].

Exercice 14.9

On considère le message suivant, que l'on va essayer de déchiffrer par force brute :

`\x1b-z4 /&a)375;3z1$z#4?r-}=/z$$/&oz\x034;<%z=/z>$z! 3&mz;-z4 /&a;$ .3 a67a9=4(3&?r%?r-?r%3 $zia+' 4`

On sait que le message contient le mot **courage**, et on sait aussi que la clef est composée de 3 lettres majuscules.

Déterminer le message en clair, ainsi que la clé utilisée.

14.2.6 Un point sur les mots de passe

En informatique, la robustesse d'un mot de passe aléatoire est exprimée en terme d'entropie de Shannon, mesurée en bits.

D'après wikipedia, « au lieu de mesurer la robustesse par le nombre de combinaisons de caractères qu'il faut tester pour trouver le mot de passe avec certitude, on utilise le logarithme en base 2 de ce nombre. Cette mesure est appelée l'entropie du mot de passe. Un mot de passe avec une entropie de 42 bits calculée de la sorte serait aussi robuste qu'une chaîne de 42 bits choisie au hasard.

En d'autres termes, un mot de passe de 42 bits de robustesse ne serait brisé de façon certaine qu'après $2^{42} = 4398046511104$ tentatives lors d'une attaque par force brute. L'ajout d'un bit d'entropie à un mot de passe double le nombre de tentatives requises, ce qui rend la tâche de l'attaquant deux fois plus difficile.»

L'entropie d'un mot de passe de taille L utilisant des caractères parmi N possibilité est donné par la formule

$$H = L \cdot \log_2(N)$$

On constate donc qu'un mot de passe de 10 caractères latin majuscules est plus «résistant» qu'un mot de passe de 6 caractères utilisant n'importe quel symbole du clavier français... Cela signifie que le nombre de caractère est nettement plus important que la diversité de ceux-ci.

Nombre de caractères	Nombres seulement	Lettres minuscules	Lettres majuscules et minuscules	Nombres, lettres majuscules et minuscules	Nombres, lettres majuscules et minuscules, symboles
4	Immédiat	Immédiat	Immédiat	Immédiat	Immédiat
5	Immédiat	Immédiat	Immédiat	Immédiat	Immédiat
6	Immédiat	Immédiat	Immédiat	Immédiat	Immédiat
7	Immédiat	Immédiat	1 seconde	2 secondes	4 secondes
8	Immédiat	Immédiat	28 secondes	2 minutes	5 minutes
9	Immédiat	3 secondes	24 minutes	2 heures	6 heures
10	Immédiat	1 minute	21 heures	5 jours	2 semaines
11	Immédiat	32 minutes	1 mois	10 mois	3 ans
12	1 seconde	14 heures	6 ans	53 ans	226 ans
13	5 secondes	2 semaines	332 années	3 000 années	15 000 ans
14	52 secondes	1 an	17 000 ans	202 000 ans	1 million d'années
15	9 minutes	27 ans	898 000 ans	12 millions d'années	77 millions d'années
16	1 heure	713 ans	46 millions d'années	779 millions d'années	5 milliards d'années
17	14 heures	18 000 ans	2 milliards d'années	48 milliards d'années	380 milliards d'années
18	6 jours	481 000 ans	126 milliards d'années	1 trillion d'années	26 trillions d'années

Combien de temps votre mot de passe résiste-t-il à une attaque ?

14.2.7 l'inconvénient du chiffrement symétrique

🔑 Le gros problème avec le chiffrement symétrique, c'est qu'il est nécessaire pour A et B de se mettre d'accord à l'avance sur la clé qui sera utilisée lors des échanges (et aussi se mettre

d'accord sur l'algorithme de chiffrement). Le chiffrement asymétrique permet d'éviter ce problème.